STL_plus

David Schwarz

2025-09-10

Read generated Data

STL Function

```
library(stlplus)
```

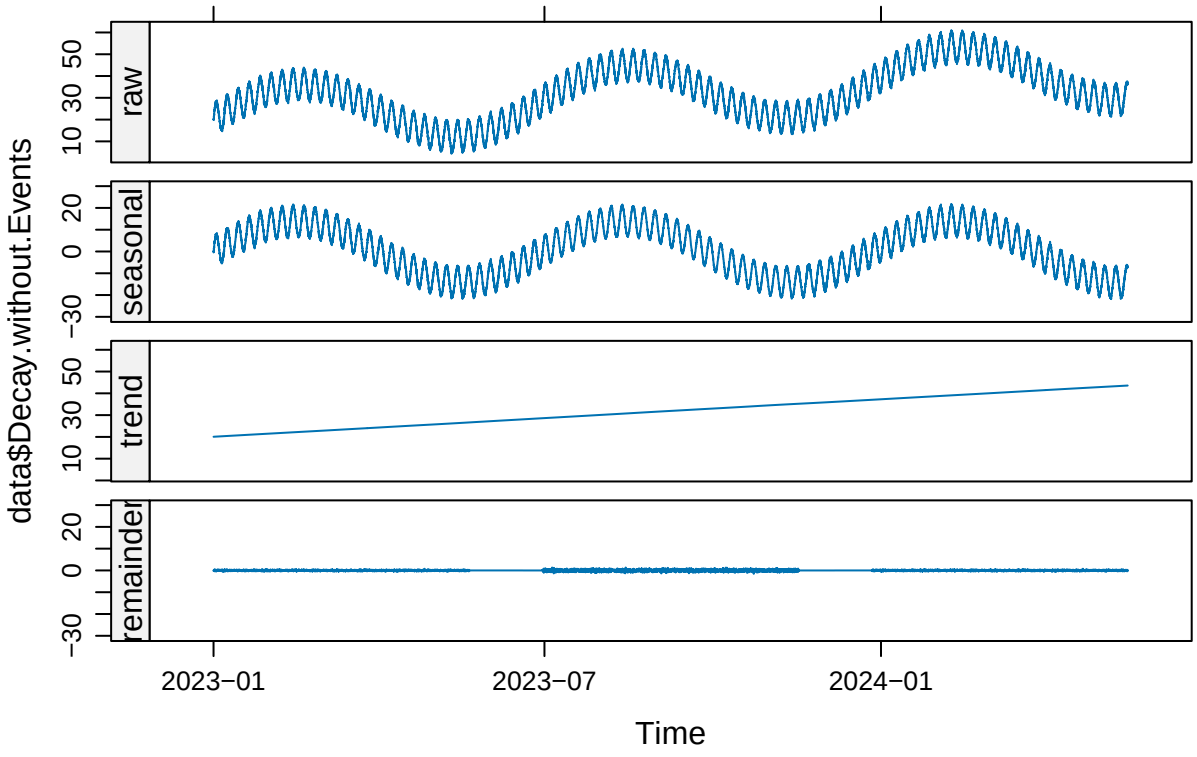
```
## Warning: package 'stlplus' was built under R version 4.4.3
```

```
stl_decompose <- function(period_days, x, time) {
  # Convert days to hours (data sample rate)
  period <- period_days * 24
  cat("using a period of", period_days, "days\n")

  # Make sure seasonal period is odd
  seasonal <- ifelse(period %% 2 == 0, period + 1, period)

  # Jumps / windows
  low_pass_jump <- floor(0.15 * (period + 1))
  seasonal_jump <- low_pass_jump
  trend_jump <- floor(0.15 * 1.5 * (period + 1))

  # STL decomposition
  stl_result <- stlplus(
    x = x,
    t = time,
    n.p = period,
    s.window = seasonal,
    l.window = low_pass_jump,
    t.window = trend_jump
  )

  # Plot result
  print(
    plot(
      stl_result,
      xlab = "Time",
      ylab = deparse(substitute(x))
    )
  )

  invisible(stl_result)
}
```

STL Decay without Events

```
res_DwoE <- stl_decompose(
  period_days = 180,
  x = data$Decay.without.Events,
  time = data$Time
)
```

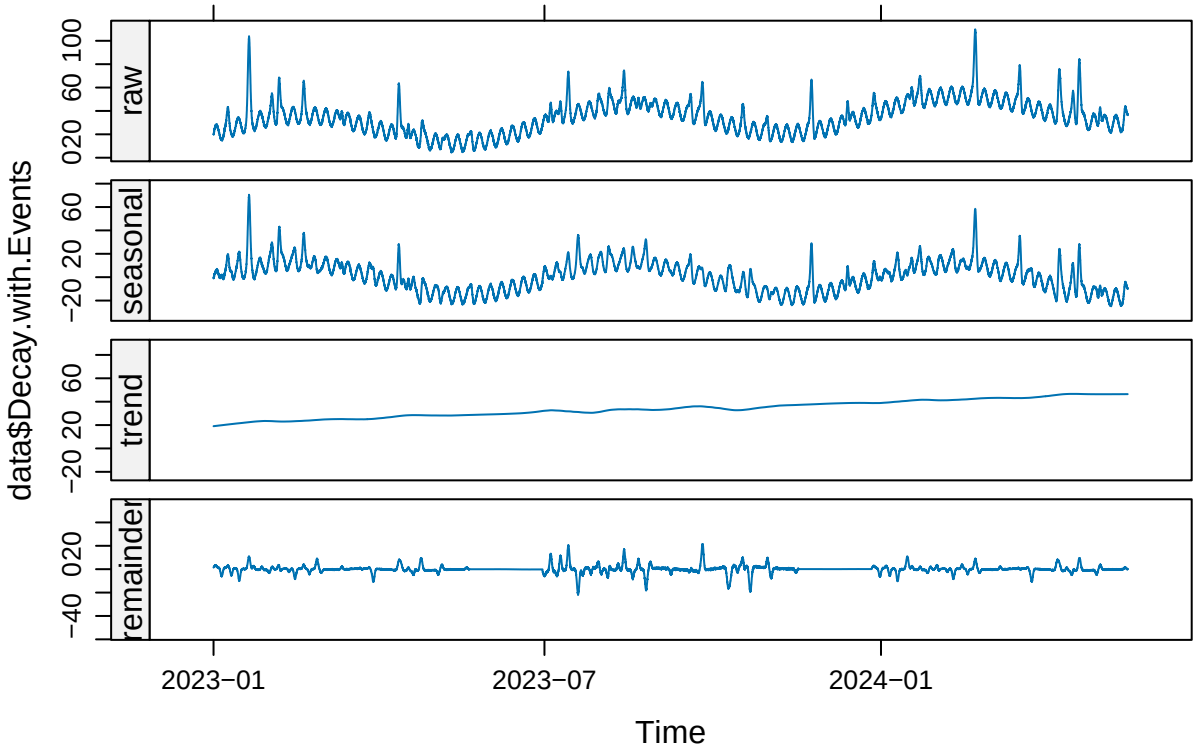
```
## using a period of 180 days
```



STL Decay with Events

```
res_DwE <- stl_decompose(
  period_days = 180,
  x = data$Decay.with.Events,
  time = data$Time
)
```

```
## using a period of 180 days
```

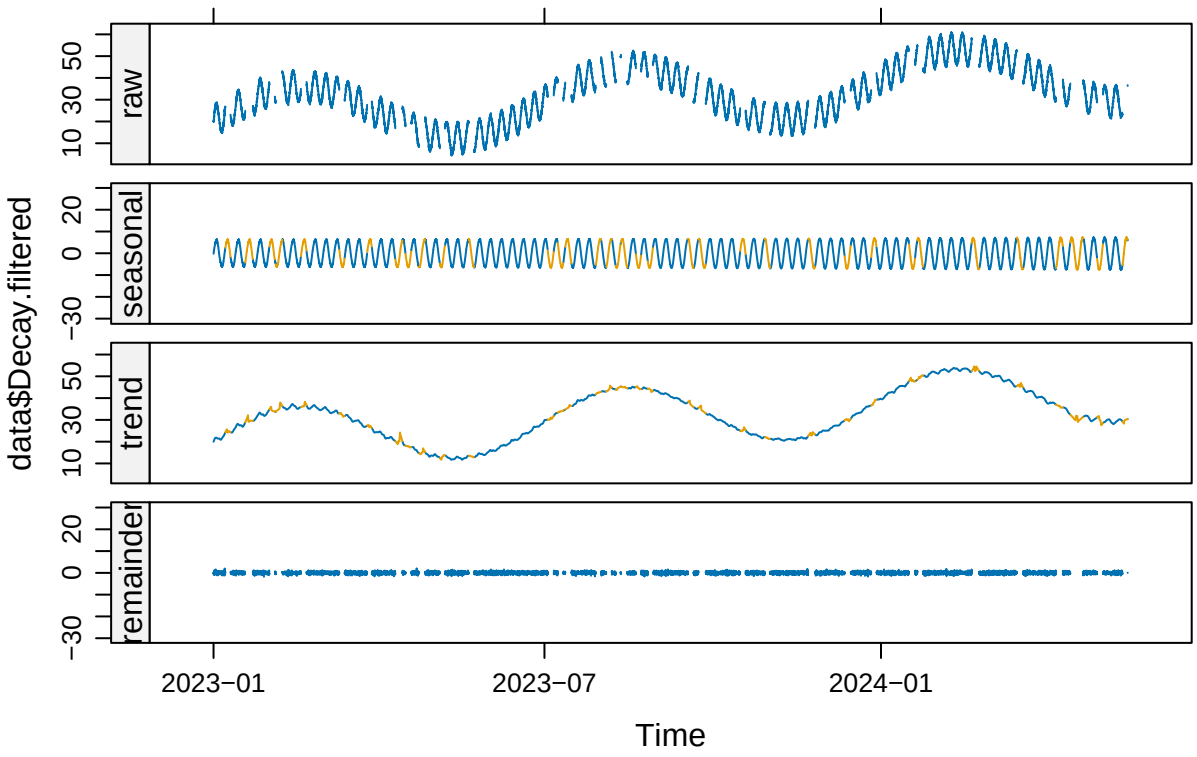


STL Decay with Gaps where Events are

Here I needed to switch to a period of 6 days as there was at least one subseries for which all values were missing when I used a period of 180 days.

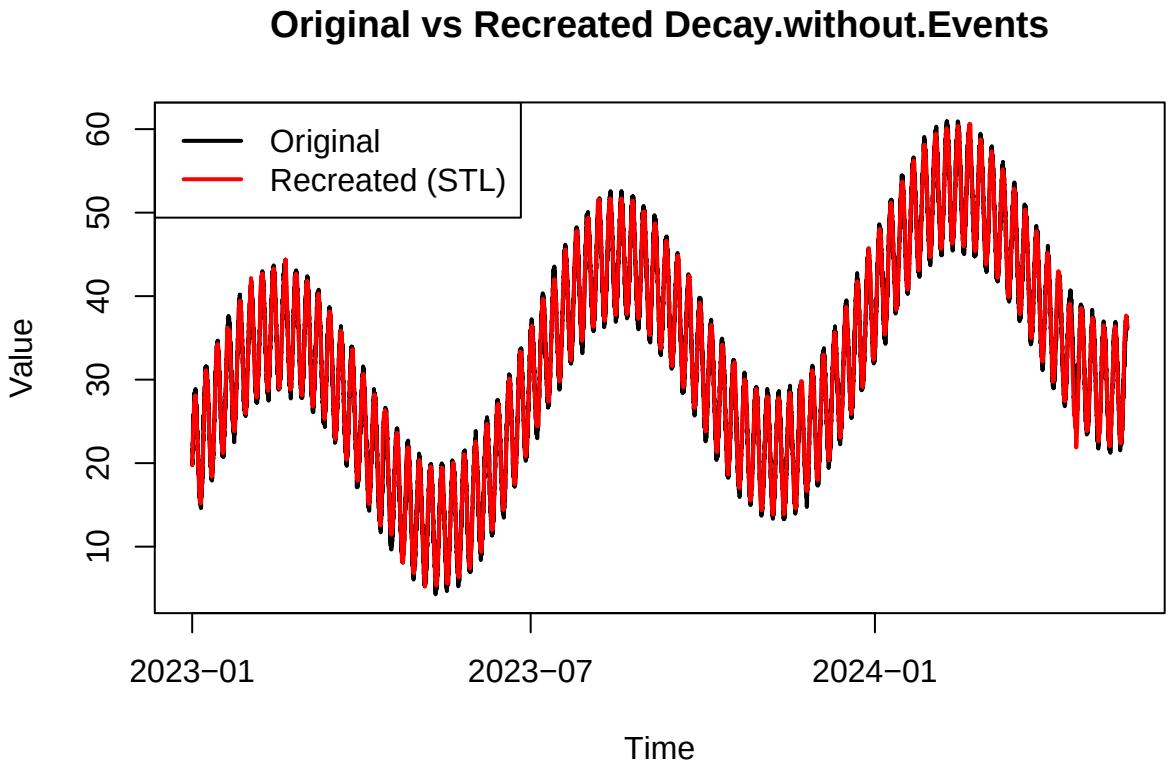
```
res_Df <- stl_decompose(
  period_days = 6,
  x = data$Decay.filtered,
  time = data$Time
)
```

```
## using a period of 6 days
```



Recreate Decay without Events

```
data$Decay.recreated <- res_Df$data$seasonal + res_Df$data$trend

orig <- data$Decay.without.Events

# Recreated (trend + seasonal)
recreated <- data$Decay.recreated

# Compare visually
plot(data$Time, orig, type = "l", col = "black", lwd = 2,
     main = "Original vs Recreated Decay.without.Events",
     xlab = "Time", ylab = "Value")
lines(data$Time, recreated, col = "red", lwd = 2)
legend("topleft", legend = c("Original", "Recreated (STL)"),
       col = c("black", "red"), lty = 1, lwd = 2)
```

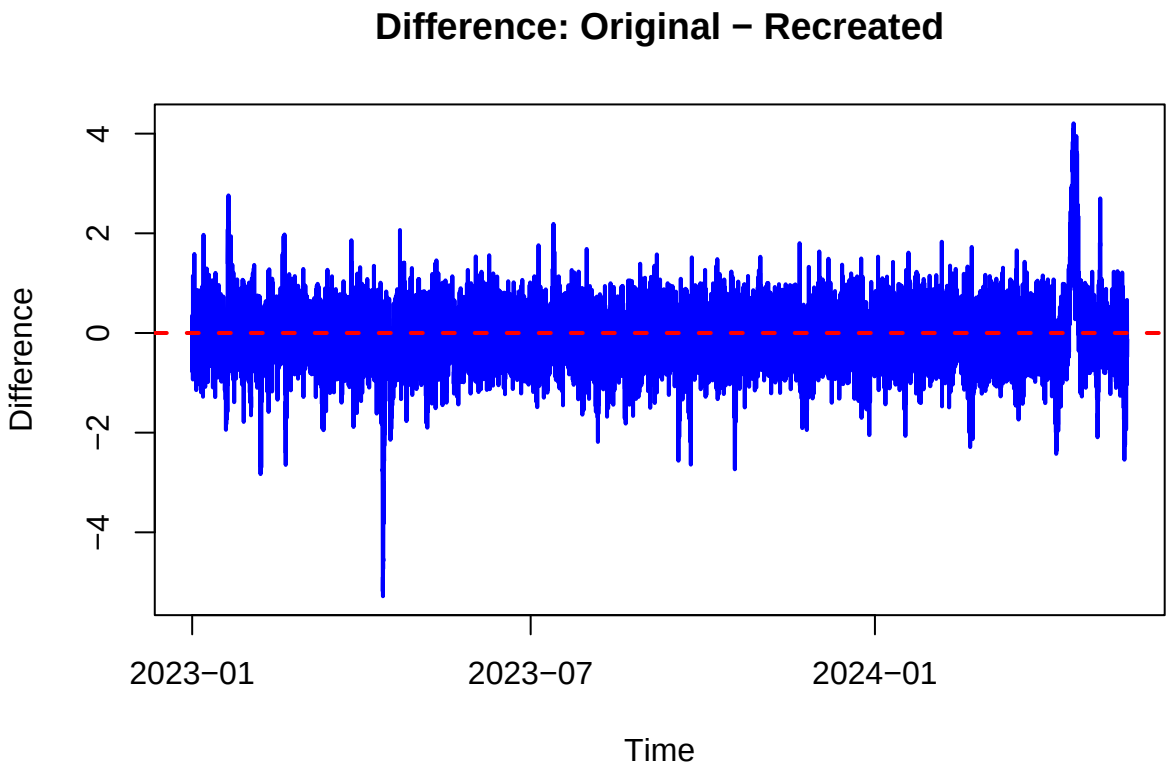



```
# Compare numerically (ignoring NAs)
mean_diff <- mean(abs(orig - recreated), na.rm = TRUE)
cat("Mean absolute difference:", mean_diff, "\n")
```

```
## Mean absolute difference: 0.4593051
```

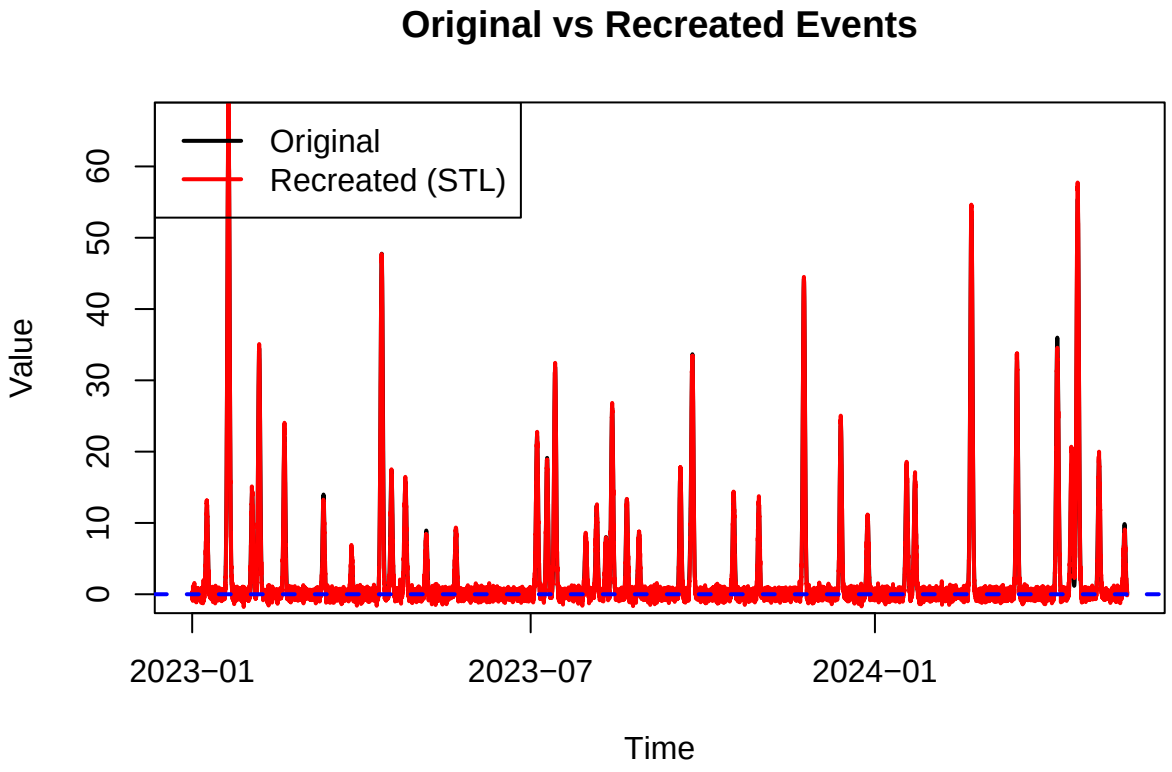
Difference between Original and Recreated

```
# Difference (residuals)
diff <- orig - recreated

# Plot the difference over time
plot(data$Time, diff, type = "l", col = "blue", lwd = 2,
     main = "Difference: Original - Recreated",
     xlab = "Time", ylab = "Difference")

abline(h = 0, col = "red", lty = 2, lwd = 2)  # reference line at 0
```



Plot showing only the events

```r
# Difference (residuals)
Events <- data$Decay.with.Events - data$Decay.without.Events
rEvents <- data$Decay.with.Events - recreated

# Compare visually
plot(data$Time, Events, type = "l", col = "black", lwd = 2,
     main = "Original vs Recreated Events",
     xlab = "Time", ylab = "Value")
lines(data$Time, rEvents, col = "red", lwd = 2)
legend("topleft", legend = c("Original", "Recreated (STL)"),
       col = c("black", "red"), lty = 1, lwd = 2)


abline(h = 0, col = "blue", lty = 2, lwd = 2)  # reference line at 0
```



Comments

In most regards the `R` code works almost the same as the `Python` code. The only difference I noticed is in the way `stlplus` handles the 'Decay with Events'. In `R` the *Seasonal* not only captures the two periods but also most of the events, where in `Python` there was much more of the events present in the *Residual*. In that way I would argue, that the `Python` version is slighlty better than the `R` version.

Regarding the mayor reason I tried `stlplus` in the first place: *" The* ***`stlplus`*** *package implements the same algorithm as the* ***`stl`*** *function in base R while allowing for missing data."*
I used this ability to reproduce the 'Decay without Events' from the 'Decay with Events' by removing all the points where the difference between the two was bigger than 0.1. For real data I would need another method to decide, where the gaps should be. The tricky part of this way of creating a dataset that only contains the events is that I need to know when events are and when not. In theory I could use the flags I have, but then I am restricting myself to the events I know.

Next steps

- try this pipeline with real data